

Atlassian MCP Server Installation Guide for OpenCode

Complete Setup for Jira and Confluence Integration

Purpose: This guide walks you through installing and configuring the official Atlassian MCP server for OpenCode, enabling AI-powered Jira and Confluence operations directly from your OpenCode sessions.

Table of Contents

- 1. [Overview](#)
 - 2. [What You'll Get](#)
 - 3. [Prerequisites](#)
 - 4. [Installation Methods](#)
 - 5. [Authentication Setup](#)
 - 6. [OpenCode Configuration](#)
 - 7. [Verification & Testing](#)
 - 8. [Available Tools](#)
 - 9. [Real-World Usage Examples](#)
 - 10. [Troubleshooting](#)
 - 11. [Best Practices](#)
 - 12. [Advanced Configuration](#)
 - 13. [Security Considerations](#)
 - 14. [FAQ](#)
-

Overview

What is the Atlassian MCP Server?

The **Atlassian MCP (Model Context Protocol) Server** is an official integration that connects OpenCode to Atlassian's APIs (Jira and Confluence), allowing you to:

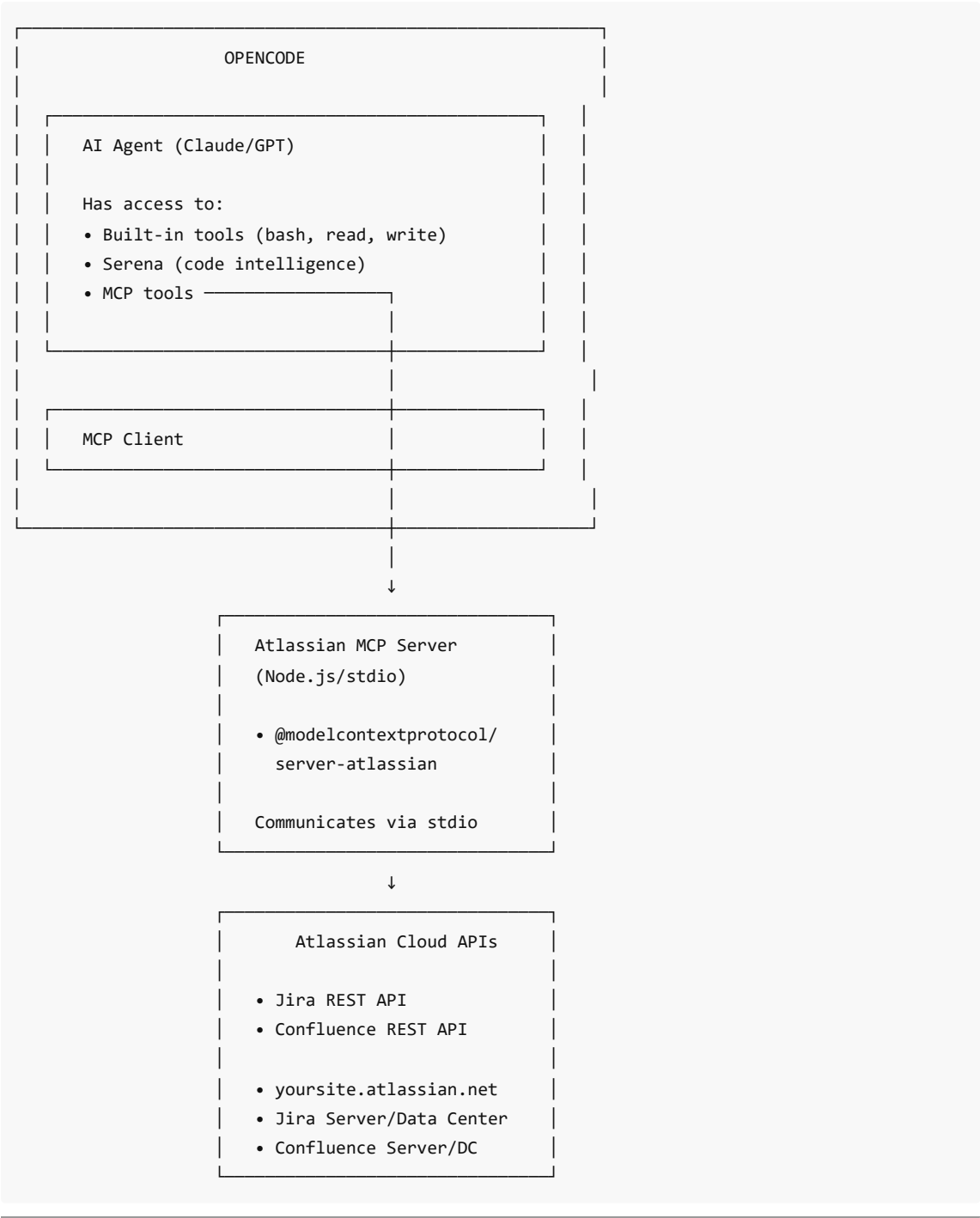
- Create, read, update, and search Jira issues
- Manage sprints, epics, and boards
- Create and edit Confluence pages
- Search Confluence content
- Add comments and attachments
- Manage workflows and transitions
- Link issues and pages
- And much more!

Why Use It?

```
WITHOUT Atlassian MCP:
You: "Create a Jira ticket for this bug"
OpenCode: "I can't access Jira. Please do it manually."

WITH Atlassian MCP:
You: "Create a Jira ticket for this bug"
OpenCode: *Analyzes code, creates detailed ticket with reproduction steps*
You: "Done! Issue PROJ-123 created and assigned."
```

Architecture



What You'll Get

Available Capabilities

Once configured, you can use natural language to:

📌 Jira Issue Management

- ☒ Create issues (Story, Bug, Task, Epic, etc.)
- ☒ Search issues (JQL queries)
- ☒ Update issue fields (status, assignee, priority, labels)
- ☒ Add comments to issues
- ☒ Add worklogs (time tracking)
- ☒ Transition issues through workflows
- ☒ Link issues (blocks, duplicates, relates to)
- ☒ Create sub-tasks
- ☒ Attach files
- ☒ Watch/unwatch issues

Jira Project Management

- ☒ List projects and issue types
- ☒ Get project metadata
- ☒ View custom fields
- ☒ Access board information
- ☒ Sprint management

Confluence Page Management

- ☒ Create pages and blog posts
- ☒ Read page content
- ☒ Update existing pages
- ☒ Search pages (CQL queries)
- ☒ Add comments (footer and inline)
- ☒ Create page hierarchies
- ☒ List spaces
- ☒ Get page descendants

Search Operations

- ☒ Advanced Jira search (JQL)
- ☒ Confluence content search (CQL)
- ☒ Rovo Search (AI-powered search across Jira & Confluence)
- ☒ Filter by projects, spaces, labels

Prerequisites

Required Software

1. **Node.js** (v18 or higher)

```
# Check your version
node --version # Should be v18.0.0 or higher
```

2. **npm** (comes with Node.js)

```
npm --version # Should be 9.0.0 or higher
```

3. **OpenCode** (v1.3.0 or higher)

```
opencode --version
```

Required Accounts

- **Atlassian Account:** Access to Jira and/or Confluence
- **Atlassian API Token:** For authentication
- **Cloud ID:** Your Atlassian site identifier

System Requirements

OS	Status	Notes
Windows	✔ Supported	Works on Windows 10/11
macOS	✔ Supported	Works on Intel and Apple Silicon
Linux	✔ Supported	All major distributions
WSL2	✔ Supported	Recommended for Windows developers

Installation Methods

Choose one of these methods based on your preference:

Method 1: NPM Global Install (Recommended)

Best for: Most users, easiest to maintain

```
# Install globally
npm install -g @modelcontextprotocol/server-atlassian

# Verify installation
which mcp-server-atlassian # macOS/Linux
where mcp-server-atlassian # Windows
```

Pros:

- ✔ Simple one-line install
- ✔ Automatic PATH setup
- ✔ Easy to update
- ✔ Works from any directory

Cons:

- ✖ Requires admin/sudo on some systems
- ✖ Global node_modules can get cluttered

Method 2: NPX (No Installation)

Best for: Testing, trying before committing, CI/CD

```
# No installation needed - npx runs on demand
# Configuration uses npx directly
```

Pros:

-  No installation step
-  Always uses latest version
-  Clean, minimal footprint

Cons:

-  Slower startup (downloads on first use)
-  Requires internet connection
-  Less control over version

Method 3: Local Project Install

Best for: Project-specific configuration, version pinning

```
# Navigate to your project
cd /path/to/your/project

# Install locally
npm install @modelcontextprotocol/server-atlassian

# Creates node_modules/@modelcontextprotocol/server-atlassian
```

Pros:

-  Version pinned in package.json
-  Team shares same version
-  No global pollution

Cons:

-  Must install per project
-  Larger project size
-  Path management in config

Recommended Choice

For most users:

```
# Just run this:
npm install -g @modelcontextprotocol/server-atlassian
```

This guide will use **Method 1 (Global Install)** for all examples.

Authentication Setup

The Atlassian MCP server requires an API token and Cloud ID for authentication.

Step 1: Find Your Cloud ID

Your Cloud ID is the subdomain of your Atlassian site.

Option A: From URL

If your Jira/Confluence URL is:

```
https://yourcompany.atlassian.net
```

Your Cloud ID is: `yourcompany.atlassian.net`

Option B: Extract from Settings

1. Go to your Jira or Confluence site
2. Click your profile icon → **Settings**
3. Look at the URL: `https://CLOUDID.atlassian.net/...`
4. The `CLOUDID` part is your Cloud ID

Examples:

-  `acme-corp.atlassian.net`
-  `my-team.atlassian.net`
-  `company123.atlassian.net`
-  `atlassian.net` (missing subdomain)
-  `https://acme.atlassian.net` (no https://)

Step 2: Create an Atlassian API Token

1. **Go to Atlassian Account Settings:**
 - Visit: <https://id.atlassian.com/manage-profile/security/api-tokens>
 - Or: Atlassian profile → Account Settings → Security → API tokens
2. **Click "Create API token"**
3. **Configure the token:**
 - **Label:** `OpenCode Atlassian MCP`
 - **Purpose:** For AI assistant integration
 - Click **Create**
4. **COPY THE TOKEN IMMEDIATELY**
 - Format: `ATATT3xFfGF0...` (starts with `ATATT`)
 - You won't see it again!

Step 3: Store the Token and Cloud ID Securely

Option A: Environment Variables (Recommended)

Windows (PowerShell):

```
# Set for current session
$env:ATLASSIAN_API_TOKEN = "ATATT3xFfGF0_your_token_here"
$env:ATLASSIAN_CLOUD_ID = "yourcompany.atlassian.net"
```

```
# Set permanently (User level)
[System.Environment]::SetEnvironmentVariable(
    'ATLASSIAN_API_TOKEN',
    'ATATT3xFfGF0_your_token_here',
    'User'
)
[System.Environment]::SetEnvironmentVariable(
    'ATLASSIAN_CLOUD_ID',
    'yourcompany.atlassian.net',
    'User'
)

# Verify
echo $env:ATLASSIAN_API_TOKEN
echo $env:ATLASSIAN_CLOUD_ID
```

Windows (CMD):

```
# Set for current session
set ATLASSIAN_API_TOKEN=ATATT3xFfGF0_your_token_here
set ATLASSIAN_CLOUD_ID=yourcompany.atlassian.net

# Set permanently
setx ATLASSIAN_API_TOKEN "ATATT3xFfGF0_your_token_here"
setx ATLASSIAN_CLOUD_ID "yourcompany.atlassian.net"

# Verify (restart CMD first)
echo %ATLASSIAN_API_TOKEN%
echo %ATLASSIAN_CLOUD_ID%
```

macOS/Linux (bash/zsh):

```
# Add to ~/.bashrc or ~/.zshrc
echo 'export ATLASSIAN_API_TOKEN="ATATT3xFfGF0_your_token_here"' >> ~/.bashrc
echo 'export ATLASSIAN_CLOUD_ID="yourcompany.atlassian.net"' >> ~/.bashrc

# Or for zsh
echo 'export ATLASSIAN_API_TOKEN="ATATT3xFfGF0_your_token_here"' >> ~/.zshrc
echo 'export ATLASSIAN_CLOUD_ID="yourcompany.atlassian.net"' >> ~/.zshrc

# Reload shell
source ~/.bashrc # or source ~/.zshrc

# Verify
echo $ATLASSIAN_API_TOKEN
echo $ATLASSIAN_CLOUD_ID
```

Option B: .env File (Project-specific)

```
# Create .env file in your project root
cat > .env << 'EOF'
ATLASSIAN_API_TOKEN=ATATT3xFfGF0_your_token_here
ATLASSIAN_CLOUD_ID=yourcompany.atlassian.net
ATLASSIAN_USER_EMAIL=your.email@company.com
EOF

# Add to .gitignore (CRITICAL!)
echo ".env" >> .gitignore
```

⚠️ **CRITICAL:** Never commit tokens to git!

Option C: Secure Credential Manager

Windows Credential Manager:

```
# Store credentials
cmdkey /generic:"atlassian-mcp-token" /user:"atlassian" /pass:"ATATT3xFfGF0_your_token"
cmdkey /generic:"atlassian-mcp-cloudid" /user:"atlassian" /pass:"yourcompany.atlassian.net"

# Retrieve (for verification)
cmdkey /list | Select-String "atlassian-mcp"
```

macOS Keychain:

```
# Store in keychain
security add-generic-password \
  -a "$USER" \
  -s "ATLASSIAN_API_TOKEN" \
  -w "ATATT3xFfGF0_your_token"

security add-generic-password \
  -a "$USER" \
  -s "ATLASSIAN_CLOUD_ID" \
  -w "yourcompany.atlassian.net"

# Retrieve from keychain
security find-generic-password \
  -a "$USER" \
  -s "ATLASSIAN_API_TOKEN" \
  -w
```

Step 4: Verify Token Works

```
# Test Jira API access
curl -u your.email@company.com:$ATLASSIAN_API_TOKEN \
  https://yourcompany.atlassian.net/rest/api/3/myself
```



```
# Expected: Your Atlassian user info in JSON
```

If you see your username and profile data, the token works!

OpenCode Configuration

Now configure OpenCode to use the Atlassian MCP server.

Configuration File Location

Choose one based on your needs:

Scope	Location	Use Case
Global	<code>~/.config/opencode/opencode.json</code>	All projects, all Atlassian sites
Project	<code>./opencode.json</code>	Current project only
Custom	<code>--config /path/to/config.json</code>	Specific config file

Recommendation: Start with **global** configuration.

Method 1: Global Install Configuration

`~/.config/opencode/opencode.json`:

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      },
      "enabled": true
    }
  }
}
```

Required Environment Variables:

- `ATLASSIAN_CLOUD_ID` - Your site domain (e.g., `yourcompany.atlassian.net`)
- `ATLASSIAN_API_TOKEN` - API token (starts with `ATATT`)
- `ATLASSIAN_USER_EMAIL` - Your Atlassian email address

Explanation:

- `"type": "stdio"` - Atlassian MCP uses standard input/output

- "command": "mcp-server-atlassian" - Assumes global install (in PATH)
- "{env:VARIABLE_NAME}" - Reads from environment variable
- "enabled": true" - Activates the server

Method 2: NPX Configuration (No Install)

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-atlassian"
      ],
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      },
      "enabled": true
    }
  }
}
```

Method 3: Local Install Configuration

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "node",
      "args": [
        "./node_modules/@modelcontextprotocol/server-atlassian/dist/index.js"
      ],
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      },
      "enabled": true
    }
  }
}
```

Windows Path Considerations

If you have spaces in your paths:

```
{
  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "C:\\Program Files\\nodejs\\node.exe",
      "args": [
        "C:\\Users\\YourUser\\AppData\\npm\\node_modules\\@modelcontextprotocol\\server-
atlassian\\dist\\index.js"
      ],
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      },
      "enabled": true
    }
  }
}
```

Note: Use double backslashes `\\` in JSON.

Complete Configuration Example

Here's a full configuration with Atlassian MCP and other settings:

~/config/opencode/opencode.json:

```
{
  "$schema": "https://opencode.ai/config.json",

  "// =====": "",
  "// GENERAL SETTINGS": "",
  "// =====": "",

  "model": "anthropic/claude-sonnet-4-5",
  "lsp": true,
  "autoupdate": true,
  "share": "manual",
  "snapshot": true,

  "// =====": "",
  "// MCP SERVERS": "",
  "// =====": "",

  "mcp": {
    "atlassian": {
      "type": "stdio",
```

```

    "command": "mcp-server-atlassian",
    "env": {
      "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
      "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
      "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
    },
    "enabled": true
  },

  "github": {
    "type": "stdio",
    "command": "mcp-server-github",
    "env": {
      "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
    },
    "enabled": true
  }
},

// =====: "",
// FILE WATCHING: "",
// =====: "",

"watcher": {
  "ignore": [
    "node_modules/**",
    "dist/**",
    "build/**",
    ".git/**",
    "*.log"
  ]
}
}

```

Verification & Testing

Step 1: Validate Configuration

```

# Check if OpenCode can load the config
opencode --validate-config

# Expected: No errors

```

Step 2: List MCP Servers

```

# List all configured MCP servers
opencode mcp list

# Expected output:

```

```
# Available MCP servers:
#   • atlassian (enabled)
#   • github (enabled)
```

Step 3: Test Connection

Start OpenCode and run these tests:

```
# Start OpenCode
opencode
```

Test 1: List tools

Show me available Atlassian tools

Expected: You should see a list of Atlassian-related tools like:

- `atlassian_createJiraIssue`
- `atlassian_searchJiraIssuesUsingJql`
- `atlassian_createConfluencePage`
- `atlassian_getConfluencePage`
- etc.

Test 2: Get current user

Who am I in Atlassian?

Expected: Your Atlassian user info

Test 3: Search Jira

Show me open issues assigned to me in Jira

Expected: List of your assigned issues

Test 4: Search Confluence

Search Confluence for "API documentation"

Expected: Search results with page titles and links

Troubleshooting Connection Issues

If tests fail, check these:

Check 1: Environment Variables are Set

```
# Windows (PowerShell)
echo $env:ATLASSIAN_API_TOKEN
echo $env:ATLASSIAN_CLOUD_ID
echo $env:ATLASSIAN_USER_EMAIL
```

```
# Windows (CMD)
echo %ATLASSIAN_API_TOKEN%
echo %ATLASSIAN_CLOUD_ID%
echo %ATLASSIAN_USER_EMAIL%

# macOS/Linux
echo $ATLASSIAN_API_TOKEN
echo $ATLASSIAN_CLOUD_ID
echo $ATLASSIAN_USER_EMAIL
```

Should show your values, not empty.

Check 2: Server is Installed

```
# Check if command exists
which mcp-server-atlassian # macOS/Linux
where mcp-server-atlassian # Windows

# Or try running directly
mcp-server-atlassian --version
```

Check 3: Token is Valid

```
# Test API access manually
curl -u $ATLASSIAN_USER_EMAIL:$ATLASSIAN_API_TOKEN \
https://$ATLASSIAN_CLOUD_ID/rest/api/3/myself
```

If this fails, your token is invalid or expired.

Check 4: OpenCode Can Start Server

```
# Check OpenCode logs
tail -f ~/.local/share/opencode/logs/mcp.log # macOS/Linux
type %LOCALAPPDATA%\opencode\logs\mcp.log    # Windows
```

Look for errors related to Atlassian MCP server startup.

Available Tools

Once configured, you have access to these Atlassian operations:

Jira Issue Management

Tool	Description	Example Usage
atlassian_createJiraIssue	Create new issue	"Create a bug for login failure"
atlassian_getJiraIssue	Get issue details	"Show me issue PROJ-123"
atlassian_editJiraIssue	Update issue	"Set PROJ-123 priority to High"

atlassian_searchJiraIssuesUsingJql	Search with JQL	"Find open bugs assigned to me"
atlassian_addCommentToJiraIssue	Add comment	"Comment on PROJ-123"
atlassian_addWorklogToJiraIssue	Log time	"Log 2 hours on PROJ-123"
atlassian_transitionJiraIssue	Change status	"Move PROJ-123 to In Progress"
atlassian_createIssueLink	Link issues	"Link PROJ-123 blocks PROJ-124"
atlassian_getTransitionsForJiraIssue	Get available transitions	"What statuses can PROJ-123 move to?"

Jira Project Management

Tool	Description	Example Usage
atlassian_getVisibleJiraProjects	List projects	"Show all Jira projects"
atlassian_getJiraProjectIssueTypesMetadata	Get issue types	"What issue types exist in PROJ?"
atlassian_getJiraIssueTypeMetaWithFields	Get field metadata	"What fields for Story in PROJ?"
atlassian_lookupJiraAccountId	Find user IDs	"Find account ID for john@example.com "

Confluence Page Management

Tool	Description	Example Usage
atlassian_createConfluencePage	Create page	"Create documentation page"
atlassian_getConfluencePage	Read page	"Show page with ID 123456"
atlassian_updateConfluencePage	Update page	"Update page 123456"
atlassian_getConfluenceSpaces	List spaces	"Show all Confluence spaces"
atlassian_getPagesInConfluenceSpace	List pages in space	"List pages in DEV space"
atlassian_getConfluencePageDescendants	Get child pages	"Show child pages of 123456"
atlassian_searchConfluenceUsingCql	Search with CQL	"Find pages with 'API' in title"

Confluence Comments

Tool	Description	Example Usage
atlassian_createConfluenceFooterComment	Add footer comment	"Comment on page 123456"
atlassian_createConfluenceInlineComment	Add inline comment	"Comment on specific text"
atlassian_getConfluencePageFooterComments	Get footer comments	"Show comments on page 123456"
atlassian_getConfluencePageInlineComments	Get inline comments	"Show inline comments"
atlassian_getConfluenceCommentChildren	Get replies	"Show replies to comment 789"

 Advanced Search

Tool	Description	Example Usage
atlassian_search	Rovo AI search	"Search for 'deployment guide'"

Real-World Usage Examples

Example 1: Create Jira Issue from Bug Report

You:

I found a critical bug: users can't log in with SSO. The error happens when clicking the SSO button on the login page. Create a Jira issue in the BACKEND project with priority High.

OpenCode:

I'll create a Jira issue for this SSO login bug.

`*Uses atlassian_createJiraIssue*`

✅ Created issue BACKEND-456 in project BACKEND:
Summary: "SSO login button not working"
Type: Bug
Priority: High
Description:
Problem
Users cannot log in using SSO authentication.

Steps to Reproduce
1. Navigate to login page
2. Click SSO login button
3. Error occurs

Impact
Critical - affects all SSO users

URL: <https://yourcompany.atlassian.net/browse/BACKEND-456>

Example 2: Search and Update Multiple Issues

You:

Find all open bugs in the MOBILE project assigned to me and add a comment saying "Working on these this sprint"

OpenCode:

Searching for your open bugs in MOBILE project...


```
*Uses atlassian_searchJiraIssuesUsingJql*
JQL: project = MOBILE AND type = Bug AND status = Open AND assignee = currentUser()
```

Found 5 issues:

- MOBILE-101: App crashes on startup
- MOBILE-102: Push notifications not working
- MOBILE-103: UI glitch on settings page
- MOBILE-104: Slow image loading
- MOBILE-105: Logout button missing

Adding comment to all 5 issues...

```
*Uses atlassian_addCommentToJiraIssue for each*
```

✅ Added comment "Working on these this sprint" to 5 issues

Example 3: Create Confluence Documentation from Code

You:

Create a Confluence page in the DEV space documenting the Authentication API.
Include the endpoints, parameters, and example requests from the auth.py file.

OpenCode:

Let me read the auth.py file first...

```
*Uses read to get file content*
```

```
*Analyzes API endpoints and parameters*
```

Creating Confluence page in DEV space...

```
*Uses atlassian_createConfluencePage*
```

✅ Created page: "Authentication API Documentation"

Content includes:

Overview

Authentication API using JWT tokens

Endpoints

POST /api/v1/auth/login

Parameters:

- email (string, required)
- password (string, required)

Response:

```
```json
```

```
{
 "token": "jwt_token_here",
```

```
"expires_in": 3600
}
```

## POST /api/v1/auth/logout

Headers:

- Authorization: Bearer {token}

## GET /api/v1/auth/verify

Verify token validity

## Example Requests

[Full examples included]

URL: <https://yourcompany.atlassian.net/wiki/spaces/DEV/pages/123456/Authentication+API+Documentation>

---

### Example 4: Link Related Issues

**\*\*You\*\*:**

Find the issue about "database migration" and link it to BACKEND-456 as "blocks"

**\*\*OpenCode\*\*:**

Searching for database migration issue...

*Uses atlassian\_searchJiraIssuesUsingJql* JQL: text ~ "database migration" AND project = BACKEND

Found: BACKEND-789 - "Database schema migration for user table"

Creating issue link...

*Uses atlassian\_createIssueLink* Link type: Blocks Inward: BACKEND-789 (blocks) Outward: BACKEND-456 (is blocked by)

✅ Linked issues: BACKEND-789 blocks BACKEND-456

Both issues now show the relationship:

- BACKEND-789: "blocks BACKEND-456"
- BACKEND-456: "is blocked by BACKEND-789"

---

### Example 5: Weekly Status Report

**\*\*You\*\*:**

Create a Confluence page with my completed tasks this week. Search for all Jira issues I resolved in the last 7 days and format them nicely.

**\*\*OpenCode\*\***:

Searching for issues you resolved this week...

*Uses atlassian\_searchJiraIssuesUsingJql* JQL: assignee = currentUser() AND status = Resolved AND resolved >= -7d

Found 12 resolved issues:

**Bugs Fixed (5):**

- BACKEND-101: Login timeout issue
- BACKEND-112: Email validation bug
- MOBILE-203: Crash on iOS 17
- MOBILE-210: Memory leak in chat
- WEB-305: CSS rendering issue

**Features Completed (4):**

- BACKEND-450: Add OAuth2 support
- WEB-321: Dark mode implementation
- MOBILE-199: Offline sync
- API-567: Rate limiting

**Tasks Done (3):**

- DOC-445: Update API docs
- TEST-234: Add integration tests
- INFRA-890: Deploy staging environment

Creating status report page...

*Uses atlassian\_createConfluencePage*

✓ Created page: "Weekly Status Report - Week of May 12, 2026" Space: STATUS

Includes:

- Summary of 12 completed items
- Breakdown by category
- Links to all issues
- Burndown chart (if available)
- Next week's priorities

URL: <https://yourcompany.atlassian.net/wiki/spaces/STATUS/pages/789012/...>

---

### Example 6: Transition Issue with Workflow

**\*\*You\*\***:

Move BACKEND-456 to "In Progress" and log 3 hours of work

**\*\*OpenCode\*\*** :

Let me check available transitions for BACKEND-456...

*Uses atlassian\_getTransitionsForJiraIssue*

Available transitions:

1. Start Progress (→ In Progress)
2. Resolve Issue (→ Resolved)
3. Close Issue (→ Closed)

Transitioning to "In Progress"...

*Uses atlassian\_transitionJiraIssue*

✓ Transitioned BACKEND-456 to "In Progress"

Logging 3 hours of work...

*Uses atlassian\_addWorklogToJiraIssue*

✓ Logged 3h on BACKEND-456 Started: 2026-05-15 09:00:00 Comment: "Investigating SSO authentication flow"

Issue status updated:

- Status: In Progress
- Time spent: 3h
- Assignee: You

---

### Example 7: Create Epic with Stories

**\*\*You\*\*** :

Create an Epic called "Mobile App Redesign" in PROJECT-A, then create 3 user stories under it:

1. Redesign home screen
2. Update navigation menu
3. Implement new color scheme

**\*\*OpenCode\*\*** :

Creating Epic...

*Uses atlassian\_createJiraIssue with issue type Epic*

✓ Created Epic: PROJECT-A-100 - "Mobile App Redesign"

Creating 3 user stories linked to the Epic...

*Uses atlassian\_createJiraIssue 3 times*

✅ Created stories:

1. PROJECT-A-101: "Redesign home screen" Type: Story Epic Link: PROJECT-A-100 Description: "Update home screen layout and components"
2. PROJECT-A-102: "Update navigation menu" Type: Story Epic Link: PROJECT-A-100 Description: "Modernize navigation with new design patterns"
3. PROJECT-A-103: "Implement new color scheme" Type: Story Epic Link: PROJECT-A-100 Description: "Apply new brand colors throughout the app"

Epic Summary:

- Epic: PROJECT-A-100
- Stories: 3
- Status: To Do
- URL: <https://yourcompany.atlassian.net/browse/PROJECT-A-100>

---

### Example 8: Search Confluence and Update Multiple Pages

**\*\*You\*\***:

Find all Confluence pages in the DEV space that mention "API v1" and add a note that "API v2 is now available"

**\*\*OpenCode\*\***:

Searching Confluence...

*Uses atlassian\_searchConfluenceUsingCql* CQL: space = DEV AND text ~ "API v1"

Found 8 pages:

1. Authentication API
2. User Management API
3. Payment Processing API
4. Webhook Integration Guide
5. API Rate Limits
6. API Changelog
7. Getting Started with APIs
8. API Troubleshooting

Would you like me to add the note to all 8 pages?

**\*\*You\*\***: `Yes, add it at the top`

**\*\*OpenCode\*\***:

*For each page: 1. Uses atlassian\_getConfluencePage to read content 2. Uses atlassian\_updateConfluencePage to prepend note*

✓ Updated 8 pages with notice:

---

**⚠ Notice: API v2 is now available with improved features and performance. Consider migrating from API v1. See the [API v2 Migration Guide](#) for details.**

Updated pages:

1. ✓ Authentication API
2. ✓ User Management API
3. ✓ Payment Processing API
4. ✓ Webhook Integration Guide
5. ✓ API Rate Limits
6. ✓ API Changelog
7. ✓ Getting Started with APIs
8. ✓ API Troubleshooting

All pages now display the API v2 notice prominently.

```

Troubleshooting

Common Issues

Issue 1: "Atlassian MCP server not found"

Symptoms:
```

Error: Cannot find module '@modelcontextprotocol/server-atlassian'

```
Solution:
```

```
```bash
# Re-install globally
npm install -g @modelcontextprotocol/server-atlassian

# Verify installation
which mcp-server-atlassian # macOS/Linux
where mcp-server-atlassian # Windows
```

Issue 2: "Authentication failed"

Symptoms:

```
Error: 401 Unauthorized
Error: Invalid credentials
```

Solutions:

Check 1: Token is set

```
echo $ATLASSIAN_API_TOKEN # Should show ATATT...
```

Check 2: Token is valid

```
curl -u $ATLASSIAN_USER_EMAIL:$ATLASSIAN_API_TOKEN \
https://$ATLASSIAN_CLOUD_ID/rest/api/3/myself
```

Check 3: Cloud ID is correct

```
echo $ATLASSIAN_CLOUD_ID # Should be yourcompany.atlassian.net
```

Check 4: Email is correct

```
echo $ATLASSIAN_USER_EMAIL # Should match your Atlassian account
```

Fix: Regenerate token and update environment variables.

Issue 3: "Cloud ID not found"

Symptoms:

```
Error: Could not find Cloud ID
Error: Site not accessible
```

Solutions:

1. Verify Cloud ID format:

```
# Correct format
ATLASSIAN_CLOUD_ID=yourcompany.atlassian.net

# Wrong formats
ATLASSIAN_CLOUD_ID=https://yourcompany.atlassian.net # Remove https://
ATLASSIAN_CLOUD_ID=yourcompany # Missing .atlassian.net
```

2. Check site accessibility:

```
curl https://$ATLASSIAN_CLOUD_ID/rest/api/3/serverInfo
```

Issue 4: "Permission denied"

Symptoms:

```
Error: 403 Forbidden
Error: You do not have permission to perform this operation
```

Solutions:

1. Check project permissions:

- Go to project settings in Jira
- Verify you have required permissions
- Ask admin to grant permissions

2. Check Confluence space permissions:

- Go to space settings
- Verify you can create/edit pages
- Ask space admin for permissions

3. Check API token scope:

- API tokens inherit user permissions
- You need appropriate role in project/space

Issue 5: "Server crashed or timeout"

Symptoms:

```
Error: MCP server atlassian failed to start
Error: Timeout waiting for server response
```

Solutions:

Check 1: Server can run manually

```
# Try running server directly
export ATlassian_CLOUD_ID="yourcompany.atlassian.net"
export ATlassian_API_TOKEN="ATATT..."
export ATlassian_USER_EMAIL="your.email@company.com"
mcp-server-atlassian

# Should start without errors
```

Check 2: Node.js version

```
node --version # Should be v18+
```

Check 3: Check logs

```
# OpenCode MCP logs
cat ~/.local/share/opencode/logs/mcp.log # macOS/Linux
type %LOCALAPPDATA%\opencode\logs\mcp.log # Windows
```

Check 4: Increase timeout

```
{
  "mcp": {
```



```

    "atlassian": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "timeout": 30000, // 30 seconds
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      }
    }
  }
}

```

Issue 6: "JQL/CQL query failed"

Symptoms:

Error: Invalid JQL query
Error: CQL syntax error

Solutions:

For JQL (Jira Query Language):

- ✗ Wrong: Find bugs assigned to me
- ✓ Correct: project = PROJ AND type = Bug AND assignee = currentUser()
- ✗ Wrong: Open issues
- ✓ Correct: status = Open
- ✗ Wrong: Issues updated today
- ✓ Correct: updated >= startOfDay()

For CQL (Confluence Query Language):

- ✗ Wrong: Find documentation
- ✓ Correct: type = page AND text ~ "documentation"
- ✗ Wrong: Pages in DEV space
- ✓ Correct: space = DEV
- ✗ Wrong: Pages created this week
- ✓ Correct: created >= now("-7d")

Test queries in Jira/Confluence first before using in OpenCode.

Issue 7: "Rate limit exceeded"

Symptoms:

Error: 429 Too Many Requests
Error: Rate limit exceeded

Solution:

```
# Check rate limit status
curl -u $ATLASSIAN_USER_EMAIL:$ATLASSIAN_API_TOKEN \
  -I https://$ATLASSIAN_CLOUD_ID/rest/api/3/myself

# Look for headers:
# X-RateLimit-Limit: 300
# X-RateLimit-Remaining: 0
# X-RateLimit-Reset: 1621234567
```

Fixes:

1. Wait for rate limit reset
2. Reduce frequency of API calls
3. Batch operations when possible
4. Consider Atlassian Cloud Premium for higher limits

Debug Mode

Enable debug logging:

```
{
  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}",
        "DEBUG": "*" // Enable all debug logs
      },
      "enabled": true
    }
  },
  "logLevel": "debug" // OpenCode debug logs
}
```

View logs:

```
# OpenCode logs
tail -f ~/.local/share/opencode/logs/*.log

# MCP server logs
tail -f ~/.local/share/opencode/logs/mcp-atlassian.log
```

Best Practices

1. Token Management

✓ DO:

- Use environment variables for tokens
- Create descriptive token labels
- Rotate tokens every 90 days
- Store in secure credential manager
- Use separate tokens for different purposes

✗ DON'T:

- Hardcode tokens in config files
 - Commit tokens to git
 - Share tokens between services
 - Use tokens in public logs
 - Leave old tokens active
-

2. JQL/CQL Queries

✓ DO:

- Test queries in Jira/Confluence UI first
- Use field names correctly
- Filter by project/space to narrow results
- Use pagination for large result sets
- Cache results when appropriate

✗ DON'T:

- Write overly complex queries
 - Fetch all data when you need one item
 - Ignore query performance
 - Use deprecated JQL functions
-

3. Content Creation

✓ DO:

- Use markdown for simple content
- Use ADF (Atlassian Document Format) for rich content
- Validate content before creating
- Use templates for consistent formatting
- Check for duplicates before creating

✗ DON'T:

- Create pages/issues without description
 - Duplicate content unnecessarily
 - Use invalid field values
 - Ignore required fields
-

4. Performance

✓ DO:

- Batch similar operations
- Use specific filters in searches
- Cache frequently accessed data
- Use pagination for large datasets
- Monitor rate limits

✗ DON'T:

- Make unnecessary API calls
- Fetch all issues/pages at once
- Make serial calls that could be parallel
- Ignore pagination

5. Security

✓ DO:

- Use HTTPS (default for Atlassian Cloud)
- Validate input before creating content
- Check permissions before operations
- Audit token usage periodically
- Revoke unused tokens

✗ DON'T:

- Disable SSL verification
- Grant excessive permissions
- Leave tokens in logs
- Use tokens in public repositories

Advanced Configuration

Multiple Atlassian Sites

You can configure multiple Atlassian instances:

```
{
  "mcp": {
    "atlassian-work": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
        "ATLASSIAN_CLOUD_ID": "work-company.atlassian.net",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_WORK_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "work@company.com"
      },
      "enabled": true
    },
    "atlassian-personal": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
```

```

    "ATLASSIAN_CLOUD_ID": "personal.atlassian.net",
    "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_PERSONAL_TOKEN}",
    "ATLASSIAN_USER_EMAIL": "personal@email.com"
  },
  "enabled": true
}
}
}

```

Usage:

Using atlassian-work: Create issue in BACKEND project
 Using atlassian-personal: Search my personal Confluence

Jira Server / Data Center

For on-premise Jira/Confluence:

```

{
  "mcp": {
    "atlassian-server": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
        "ATLASSIAN_CLOUD_ID": "jira.company.local",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_SERVER_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "user@company.com",
        "ATLASSIAN_API_BASE_URL": "https://jira.company.local"
      },
      "enabled": true
    }
  }
}

```

Proxy Configuration

If behind a corporate proxy:

```

{
  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}",
        "HTTP_PROXY": "http://proxy.company.com:8080",
        "HTTPS_PROXY": "http://proxy.company.com:8080",
        "NO_PROXY": "localhost,127.0.0.1"
      }
    }
  }
}

```

```
    },
    "enabled": true
  }
}
```

Custom Timeouts

For slow networks or large operations:

```
{
  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "timeout": 60000, // 60 seconds (default: 30s)
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      },
      "enabled": true
    }
  }
}
```

Security Considerations

1. Token Storage

Security Levels (Best to Worst):

1. ★★★★★ **Secure Credential Manager**

- macOS Keychain
- Windows Credential Manager
- Linux Secret Service

2. ★★ **Environment Variables (User-level)**

- Stored in shell RC files
- Not visible to other users
- Persists across sessions

3. ★ **Environment Variables (Session-level)**

- Set for current session only
- Lost on session end
- Good for temporary use

4. ✖ **Project .env files**

- Risk of accidental commit

- Visible to anyone with file access
- MUST be in .gitignore

5. ❌❌ Hardcoded in config

- NEVER do this
 - Will be committed to git
 - Visible in version history
-

2. Token Permissions

Atlassian API tokens have the same permissions as the user. Best practices:

- ✅ Create dedicated service accounts for automation
 - ✅ Grant minimum necessary permissions
 - ✅ Use different tokens for different purposes
 - ✅ Document what each token is used for
 - ❌ Don't use admin accounts for automation
 - ❌ Don't grant unnecessary project access
-

3. Token Rotation

Recommended Schedule:

- Personal use: Every 90 days
- Team automation: Every 60 days
- Production systems: Every 30 days

Rotation Process:

1. Create new token
 2. Update environment variable
 3. Test new token works
 4. Revoke old token
 5. Update documentation
-

4. Monitoring

What to Monitor:

- API rate limit usage
- Failed authentication attempts
- Unusual access patterns
- Token usage from unexpected locations

How to Monitor:

- Check Atlassian audit logs regularly
 - Set up alerts for failed authentications
 - Review API usage statistics
 - Monitor for suspicious activity
-

5. Incident Response

If token is compromised:

1. Immediate (within 5 minutes):

- Revoke token at <https://id.atlassian.com/manage-profile/security/api-tokens>
- Remove from environment variables
- Remove from config files

2. Short-term (within 1 hour):

- Generate new token
- Update configurations
- Check Atlassian audit logs
- Review recent API calls
- Check for unauthorized changes in Jira/Confluence

3. Follow-up (within 24 hours):

- Review security logs
- Update documentation
- Implement better storage
- Consider enabling 2FA
- Notify security team if needed

FAQ

Q: Can I use username/password instead of API token?

A: No. Basic authentication with username/password is deprecated by Atlassian. You must use API tokens.

Q: Does this work with Jira Server / Data Center?

A: Yes! Set the `ATLASSIAN_API_BASE_URL` environment variable to your server URL. See [Advanced Configuration](#).

Q: Does this work with Confluence Server / Data Center?

A: Yes! Same as Jira Server - configure the base URL appropriately.

Q: Can I use this for multiple Atlassian sites?

A: Yes! Configure multiple MCP instances with different names and credentials. See [Multiple Atlassian Sites](#).

Q: Will this create issues/pages without asking me?

A: OpenCode will always explain what it's going to do before executing operations. You can review and approve/reject. Configure confirmation requirements in your OpenCode settings.

Q: Can I use this for organization projects?

A: Yes, if you have appropriate permissions in those projects/spaces.

Q: Does this work offline?

A: No. The Atlassian MCP server requires internet access to communicate with Atlassian APIs.

Q: Can I limit which projects OpenCode can access?

A: Use a service account with limited project permissions, or create separate tokens for different projects.

Q: How do I update the Atlassian MCP server?

```
# For global install
npm update -g @modelcontextprotocol/server-atlassian

# Check version
npm list -g @modelcontextprotocol/server-atlassian
```

Q: Can I use this with both Jira and Confluence?

A: Yes! One API token works for both. The MCP server provides tools for both Jira and Confluence operations.

Q: What's the difference between JQL and CQL?

A:

- **JQL (Jira Query Language):** Search Jira issues
 - **CQL (Confluence Query Language):** Search Confluence pages
-

Q: Can I create custom fields in Jira?

A: Yes, but you need to know the custom field IDs. Use `atlassian_getJiraIssueTypeMetaWithFields` to discover available fields and their IDs.

Q: How do I find my Cloud ID?

A: It's the subdomain in your Atlassian URL. For `https://acme.atlassian.net`, the Cloud ID is `acme.atlassian.net`.

Q: What happens if my rate limit is exceeded?

A: Operations will fail with a 429 error. Wait for the rate limit to reset (check response headers for reset time) or reduce the frequency of API calls.

Q: Can I use this with the Atlassian CLI (`jira` command)?

A: Yes, they don't conflict. The MCP server is for AI integration, while the CLI is for direct command-line use. Both can coexist.

Summary Checklist

Installation

- ☐ Node.js v18+ installed
- ☐ Atlassian MCP server installed: `npm install -g @modelcontextprotocol/server-atlassian`
- ☐ Installation verified: `mcp-server-atlassian --version`

Authentication

- ☐ API Token created on Atlassian
- ☐ Cloud ID identified (e.g., `yourcompany.atlassian.net`)
- ☐ User email noted
- ☐ Token stored securely in environment variables
- ☐ Token tested: `curl` request works

Configuration

- ☐ OpenCode config file created/updated
- ☐ MCP server configured with correct command
- ☐ Environment variables referenced correctly
- ☐ Server enabled: `"enabled": true`

Verification

- ☐ Config validated: `opencode --validate-config`
- ☐ Server listed: `opencode mcp list`
- ☐ Tools available in OpenCode session
- ☐ Test operation successful (e.g., search issues)

Security

- ☐ Token not hardcoded in config
- ☐ Token not committed to git
- ☐ Token rotation scheduled
- ☐ .gitignore includes sensitive files
- ☐ Appropriate permissions granted

Quick Start (TL;DR)

For the impatient, here's the fastest path:

```
# 1. Install
npm install -g @modelcontextprotocol/server-atlassian

# 2. Create API token at:
# https://id.atlassian.com/manage-profile/security/api-tokens

# 3. Set environment variables (pick your shell)
# PowerShell:
$env:ATLASSIAN_CLOUD_ID = "yourcompany.atlassian.net"
$env:ATLASSIAN_API_TOKEN = "ATATT3xFfGF0_your_token"
```

```
$env:ATLASSIAN_USER_EMAIL = "your.email@company.com"

# Bash/Zsh:
export ATLASSIAN_CLOUD_ID="yourcompany.atlassian.net"
export ATLASSIAN_API_TOKEN="ATATT3xFfGF0_your_token"
export ATLASSIAN_USER_EMAIL="your.email@company.com"

# 4. Configure OpenCode
echo '{
  "mcp": {
    "atlassian": {
      "type": "stdio",
      "command": "mcp-server-atlassian",
      "env": {
        "ATLASSIAN_CLOUD_ID": "{env:ATLASSIAN_CLOUD_ID}",
        "ATLASSIAN_API_TOKEN": "{env:ATLASSIAN_API_TOKEN}",
        "ATLASSIAN_USER_EMAIL": "{env:ATLASSIAN_USER_EMAIL}"
      },
      "enabled": true
    }
  }
}' > ~/.config/opencode/opencode.json

# 5. Test
opencode
# Then type: "Show me open issues assigned to me"
```

Done! 🎉

Additional Resources

Official Documentation

- **Atlassian MCP Server:** <https://github.com/modelcontextprotocol/servers/tree/main/src/atlassian>
- **MCP Specification:** <https://modelcontextprotocol.io/>
- **OpenCode Docs:** <https://opencode.ai/docs>
- **Jira REST API:** <https://developer.atlassian.com/cloud/jira/platform/rest/v3/>
- **Confluence REST API:** <https://developer.atlassian.com/cloud/confluence/rest/v2/>

Community

- **OpenCode Discord:** <https://discord.gg/opencode>
- **Atlassian Community:** <https://community.atlassian.com/>
- **MCP Discussions:** <https://github.com/modelcontextprotocol/servers/discussions>

Related Guides

- **OpenCode Installation Guide** - How to install OpenCode
- **Serena Code Intelligence Guide** - Using OpenCode's built-in code tools
- **GitHub MCP Installation Guide** - GitHub integration setup

Document Information:

- **Version:** 1.0
 - **Last Updated:** May 15, 2026
 - **OpenCode Compatibility:** 1.3.0+
 - **Atlassian MCP Version:** Latest (check npm)
 - **Author:** OpenCode Documentation Team
-

Happy coding with Atlassian and OpenCode! 🚀

If you have questions or run into issues, refer to the [Troubleshooting](#) section or ask in the OpenCode community.